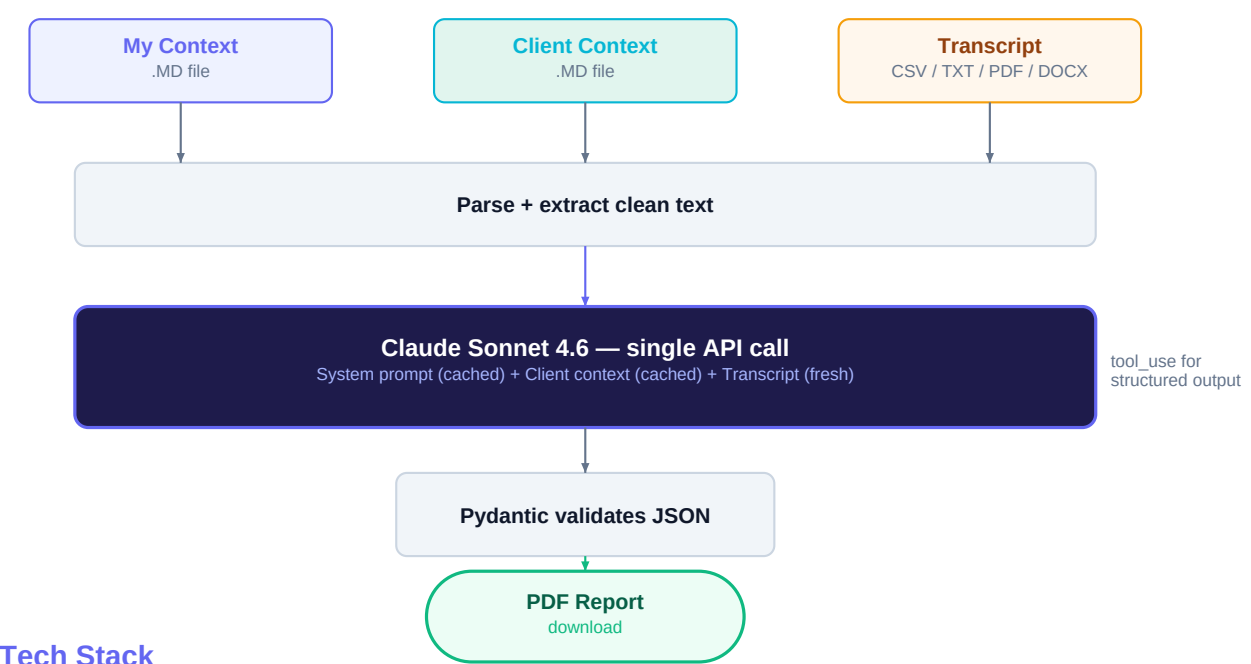


Meeting Intelligence Chatbot

System Design Brief — No RAG · No Database · Ship-Today Architecture

A document processing pipeline that ingests three inputs — your context, your client's context, and a meeting transcript — and produces a structured, branded PDF report with outcomes, action items, key dates, discussion points, and per-participant tasks. Single Claude API call. No vector database. No RAG. No persistent storage.

Architecture



Tech Stack

Layer	Technology	Notes
LLM	Claude Sonnet 4.6	1M context window (GA, no surcharge) · tool_use for structured output
Backend	FastAPI (Python)	Single endpoint · synchronous pipeline
Frontend	Next.js + Tailwind	3-zone upload UI · animated processing states
PDF Output	ReportLab	Open source · branded, styled PDF generation
File Parsing	pypdf + python-docx + csv	Open source · all transcript formats supported
Hosting	Railway (2 services)	Backend ~512MB · Frontend ~256MB · ~\$10/mo
Database	None	Stateless pipeline — no persistence needed
Vector DB	None	All docs fit in Claude's 1M context window (GA, no surcharge)

Total dependencies: fastapi · uvicorn · anthropic · reportlab · python-docx · pypdf · python-multipart · pydantic-settings — 8 packages, ~200MB Docker image

How Each Input Document Is Used

Document	Content	Role in Claude Prompt	Caching
My Context (.MD)	Your profile, company, role, style	System prompt — sets report tone	Cached (90% off)
Client Context (.MD)	Client profile, goals, ongoing work	Context block — adds background	Cached per client
Meeting Transcript	Actual meeting (CSV/TXT/PDF/DOCX)	Message — source of truth	Fresh every call

My Context + Client Context together make the report **context-aware**: Claude knows Alice is the client's CTO, knows which project "the proposal" refers to, and frames action items from your organisation's perspective.

Prompt Caching Strategy

Prompt Section	Cached?	Cost Effect	TTL
System prompt + My Context	Yes	First call 1.25x, subsequent 0.1x	5 min
Client Context	Yes	Cached per-client, 90% off on reuse	5 min
Meeting Transcript	No	Full price every call (always new)	N/A

Batch workflow: processing multiple meetings for the same client back-to-back maximizes cache hits. Both the system prompt and client context stay warm for 5 minutes, so calls 2+ pay 90% less on those tokens. Implementation requires adding `cache_control: {"type": "ephemeral"}` to each cached block — two lines of code.

PDF Report Output Sections

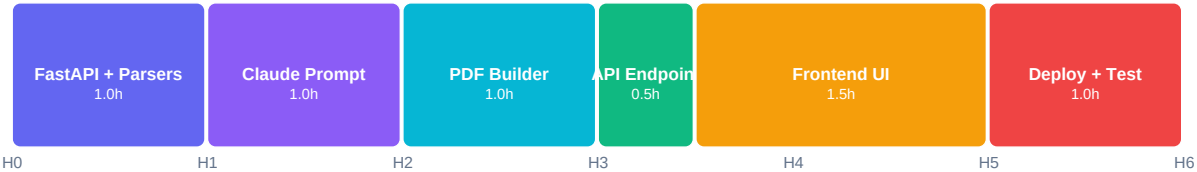
Section	Detail	Source
Executive Summary	3-4 sentence overview, framed from your perspective	Full transcript
Outcomes	Decisions: decided / pending / tabled + owner	Full transcript
Action Items	Task + assignee + due date + priority (high/med/low)	Full transcript
Key Dates	Deadlines, milestones, follow-ups with type tag	Full transcript
Discussion Points	Topic + summary + participants + resolution	Full transcript
Per-Participant Tasks	Per-person: role, assigned tasks, contributions	Transcript + client .MD
Risks and Blockers	Identified risks or impediments from discussion	Full transcript

Claude uses `tool_use` (structured output) instead of raw JSON, guaranteeing valid schema every time. Pydantic validates the response before PDF generation.

Why No RAG?

Claude's 1M context fits all 3 documents for any real-world meeting (up to ~50 hrs of continuous speech). RAG hurts report quality: it silently drops content, breaks speaker attribution, and loses narrative threads across chunks. For *comprehensive extraction* (not Q&A;), direct context gives 100% recall. Simpler, faster, cheaper, more accurate.

Shipping Sprint — 6 Hours



Hour	Task	Build	Skip for Today
H1	Backend + Parsers	FastAPI skeleton · CSV/TXT/PDF/DOCX/MD parsers	—
H2	Claude Integration	Prompt builder · tool_use schema · extractor	Two-pass extraction
H3	PDF Builder	ReportLab styled output · all 7 report sections	Complex branding
H3.5	API Wiring	POST /generate-report · file validation · errors	Auth, rate limit
H4-5	Frontend UI	3 upload zones · generate button · download card	Fancy animations
H6	Railway Deploy	Docker build · env vars · end-to-end smoke test	CI/CD pipeline

Post-Ship Roadmap

When	Feature	Effort	Impact
Week 1	Meeting type selector (standup/planning/review/client)	30 min	Better tailored reports
Week 1	Two-pass extraction (facts then synthesis)	1 hour	Higher recall, fewer missed items
Week 1	Prompt caching optimization	30 min	10-15% cost reduction
Week 2	Context file persistence (localStorage or DB)	2 hours	No re-uploading .MD files
Week 2	Markdown + text export alongside PDF	1 hour	Copy-paste into project tools
Week 2	Report comparison (diff two meetings, same client)	2 hours	Track progress across meetings
Week 3	Framer Motion animations + polished UX	3 hours	Production-quality frontend
Week 3	Batch processing (multiple transcripts at once)	3 hours	Process meeting backlog

Railway Deployment

Service	RAM	Docker Image	Monthly Cost
backend (FastAPI)	512 MB	~200 MB	~\$5-7
frontend (Next.js)	256 MB	~150 MB	~\$3-5

Total infrastructure: Railway ~\$10/mo + Claude API pay-per-use. No vector DB. No managed database. No Redis. Two services, zero operational overhead.